Research Report: GPU-based morphological operations using PyTorch F.max_pool2d for erosion and dilation vs OpenCV cv2.erode cv2.dilate quality comparison. Is max_pool2d on inverted mask equivalent to cv2.erode with square kernel? What about circular/disk structuring elements? F.avg_pool2d vs Gaussian blur for alpha feathering quality. Also: CUDA pinned memory (page-locked) for GPU to CPU transfers - measured performance gains, memory overhead, gotchas in production ML inference pipelines.

Generated: 03/12/2026, 19:19:54
Sources: 103 verified

Executive Summary

GPU-based morphological operations in PyTorch offer substantial performance improvements over CPU-based OpenCV but introduce critical quality trade-offs. For erosion with a square kernel, PyTorch's `F.max_pool2d` applied to an inverted binary mask is a direct and high-performance equivalent to OpenCV's `cv2.erode`. However, this method fails to create true circular structuring elements, instead producing an octagonal approximation that can distort non-rectangular shapes. For alpha feathering, the GPU-native `F.avg_pool2d` (box blur) is faster but produces blocky artifacts, making the smoother result from `cv2.GaussianBlur` or a pure-GPU alternative like Kornia's `filters.gaussian_blur2d` qualitatively superior.

Optimizing the required GPU-to-CPU data transfers with CUDA pinned (page-locked) memory can nearly double I/O throughput, with benchmarks showing effective bandwidth increasing from 6-8 GB/s to 12-14 GB/s and transfer times for a specific workload dropping from 54ms to 27ms [30, 25]. However, this introduces a significant production risk. Because pinned memory cannot be swapped to disk, over-allocation in multi-process inference servers can exhaust physical RAM, forcing the OS to swap other critical processes and causing system-wide "thrashing" and performance degradation. This state is best diagnosed by monitoring for consistently non-zero swap-in (`si`) and swap-out (`so`) values in `vmstat` [41, 42].

Key Findings

1. Finding: Using `torch.nn.functional.max_pool2d` on an inverted binary mask is functionally identical to `cv2.erode` with a square structuring element, providing a significant performance boost by keeping the operation on the GPU.

Confidence: MEDIUM
Sources: Multiple sources

2. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

3. Sources: [19, 23]

Confidence: MEDIUM
Sources: Multiple sources

4. Finding: Approximating a circular structuring element with multiple `F.max_pool2d` passes results in a lower-quality, octagonal shape that is anisotropic (non-uniform), whereas a custom CUDA kernel can achieve a perfectly accurate isotropic circle.

Confidence: LOW
Sources: Multiple sources

5. Confidence: MEDIUM

Confidence: MEDIUM
Sources: Multiple sources

6. Sources: [33, 34, 43]

Confidence: MEDIUM
Sources: Multiple sources

7. Finding: For alpha matte feathering, `F.avg_pool2d` creates a box blur with visible artifacts, while `cv2.GaussianBlur` or pure-GPU equivalents like `kornia.filters.gaussian_blur2d` produce a qualitatively superior smooth, natural falloff.

Confidence: MEDIUM
Sources: Multiple sources

8. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

Detailed Analysis

Morphological Operation Equivalence and Quality

Erosion/Dilation with Square Kernels
For binary masks, morphological dilation can be implemented directly on the GPU using `torch.nn.functional.max_pool2d`. The operation is identical to `cv2.dilate` when using a rectangular structuring element (`MORPH_RECT`) of the same size as the pooling kernel [19, 23].

To perform erosion, which is the dual of dilation, the input mask must first be inverted (0s become 1s, 1s become 0s). After applying `F.max_pool2d` to the inverted mask, the result is inverted back. This three-step process perfectly replicates the behavior of `cv2.erode` with a square kernel while keeping all computations on the GPU, thus avoiding costly data transfers [19]. The performance advantage of this GPU-native approach over a CPU-based OpenCV call increases with image size and kernel size [31].

Approximating Circular Structuring Elements
While `F.max_pool2d` is effective for square kernels, it cannot natively create true circular or elliptical structuring elements, which are often preferred for cleaning up alpha mattes on organic shapes to ensure uniform erosion or dilation [33]. A common workaround is to chain multiple `max_pool2d` calls with different kernel shapes (e.g., a 3×1 followed by a 1×3), which produces an approximation of a circle, such as an octagon [34].

This approximation has a significant quality drawback: it is anisotropic, meaning the erosion or dilation effect is not uniform in all directions. This can lead to visible artifacts and distortion. A custom CUDA kernel offers a qualitatively superior solution by allowing for an explicit distance check from the kernel's center, implementing a true, isotropic circular element [43]. While no direct benchmarks measuring the Intersection over Union (IoU) were found, a custom kernel would by definition achieve a perfect IoU of 1.0 against a digital circle, while the `max_pool2d` approximation would not. Furthermore, a custom kernel can fuse the entire operation into a single launch, minimizing memory latency and likely outperforming the multiple sequential launches required by the pooling approximation [44].

Alpha Feathering: Box Blur vs. Gaussian Blur
For feathering the edges of an alpha matte, a smooth falloff is critical for a natural-looking composite. PyTorch's `F.avg_pool2d` performs a box blur, where each output pixel is the unweighted average of its neighbors in a rectangular window. This method is computationally fast on the GPU but tends to produce visible blocky or rectangular artifacts, which are undesirable for high-quality results [20, 37].

In contrast, `cv2.GaussianBlur` uses a kernel where pixel weights follow a Gaussian distribution, giving more importance to central pixels and creating a much smoother, more natural blur [37]. While this requires a `GPU->CPU->GPU` roundtrip, the quality is far superior. The best-of-both-worlds solution is to use a pure-GPU Gaussian blur implementation, such as `kornia.filters.gaussian_blur2d` or `torchvision.transforms.GaussianBlur`. These functions provide the high quality of a true Gaussian blur while executing entirely on the GPU, avoiding both the performance penalty of a CPU roundtrip and the quality issues of a box blur approximation [46].

Optimizing GPU-CPU Data Transfers with Pinned Memory

Performance Gains
When data is transferred from CPU to GPU (or vice-versa), standard pageable memory must first be copied by the CPU into a special staging buffer of pinned (or page-locked) memory before the GPU's DMA engine can begin the transfer [29, 30]. By allocating memory as pinned from the start (`pin_memory=True` in PyTorch's `DataLoader`), this extra CPU-side copy is eliminated, allowing the DMA engine to access the host memory directly.

This optimization can nearly double the effective data transfer bandwidth. Benchmarks show throughput on a PCIe 4.0 system increasing from a typical 6-8 GB/s for pageable memory to 12-14 GB/s for pinned memory [30]. In practice, this can translate to significant end-to-end performance gains in I/O-bound workloads, with users reporting training loop speedups of 30-50% [25, 26]. One specific benchmark showed that using pinned memory with an asynchronous transfer reduced the time to move a batch of data from 54ms to just 27ms [25].

Production Risks and Gotchas
The primary drawback of pinned memory is that it is a scarce resource that the operating system cannot page out to disk [11, 40]. In a multi-process ML inference server or a PyTorch `DataLoader` with `num_workers > 0`, this can become a critical issue. When `pin_memory=True` is enabled, each worker process allocates its own separate pool of pinned memory. The total system-wide allocation is therefore multiplied by the number of workers, which can quickly exhaust the available physical RAM [11, 12].

When the amount of unevictable pinned memory becomes too large, the OS is forced to swap other applications and even its own critical processes to disk to free up RAM. This state, known as "thrashing," leads to massive I/O wait times and can cause the entire system to slow down dramatically or freeze [41]. The overhead of allocating pinned memory (`cudaHostAlloc()`) is also significantly higher than a standard `malloc()`, which can add latency during worker initialization [40].

Monitoring and Diagnosis
The most direct indicators of system thrashing caused by pinned memory over-allocation are found in the `vmstat` utility.

- **`si` (swap-in) and `so` (swap-out):** These columns show the amount of memory being swapped from and to the disk. If these values are consistently non-zero, it is a strong sign that the system is under memory pressure and is thrashing [41, 42].

- `wa` **(I/O wait):** A high value here indicates the CPU is spending significant time waiting for I/O operations (like swapping) to complete.

Concurrently, checking `/proc/meminfo` for the `Unevictable` value will confirm the root cause. A large `Unevictable` size confirms that a significant portion of physical RAM is locked and cannot be managed by the OS, forcing it to swap other pageable memory [45].

Recommendations

1. **For square/rectangular morphology, use `F.max_pool2d` on the GPU.** For erosion, apply it to an inverted mask. This provides a fast, high-performance equivalent to OpenCV without CPU data transfers.
2. **For high-quality feathering or circular morphology, use a pure-GPU library like Kornia.** Functions like `kornia.filters.gaussian_blur2d` provide superior quality to pooling approximations and are faster than a `GPU->CPU->OpenCV->CPU->GPU` roundtrip, even one optimized with pinned memory.
3. **Use pinned memory (`pin_memory=True`) for single-process inference or during development.** This is a safe and effective way to accelerate data loading by maximizing GPU-CPU transfer bandwidth when memory usage is predictable and controlled.
4. **In multi-process production servers, disable pinned memory by default or profile RAM usage meticulously.** The risk of memory multiplication (`num_workers` * batch size) leading to system-wide thrashing is high. If enabled, monitor system health with `vmstat` and `/proc/meminfo` to ensure `si` / `so` remain at zero and `Unevictable` memory stays within reasonable limits.
5. **Consider custom CUDA kernels for complex, repetitive post-processing.** For pipelines requiring multiple steps like circular erosion followed by a blur, fusing these into a single CUDA kernel can offer the highest possible performance by minimizing kernel launch overhead and memory bandwidth usage.

Limitations & Caveats

- The analysis lacks direct, published benchmarks comparing the Intersection over Union (IoU) and latency of a multi-pass `F.max_pool2d` octagonal approximation against a custom CUDA kernel for circular erosion. The conclusion of the custom kernel's superiority is based on established principles of GPU computing (fused kernels, algorithmic accuracy).

- The performance gains cited for pinned memory are illustrative and highly dependent on the specific hardware configuration (e.g., PCIe generation, CPU/GPU models, RAM speed) and the size of the data being transferred.

- The research primarily focuses on PyTorch and OpenCV. Other deep learning frameworks or image processing libraries may have different implementations, performance characteristics, and best practices.

Follow-up Questions

1. What is the measured IoU and latency difference between a multi-pass `F.max_pool2d` octagonal approximation and a custom CUDA circular erosion kernel on a standard dataset of alpha mattes?
2. How does the performance of Kornia's `filters.gaussian_blur2d` scale with kernel size and image resolution compared to a `GPU->CPU->cv2.GaussianBlur->CPU->GPU` roundtrip using pinned memory?
3. At what combination of `num_workers` and batch size does the total pinned memory allocation typically exceed a safe threshold (e.g., 25%) of system RAM on a standard cloud inference instance?
4. Can `torch.compile` effectively fuse a Python function that chains multiple `F.max_pool2d` calls, and how does its performance compare to a single, hand-written CUDA kernel for the same task?

Sources

[1] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFP5zHuR4uGQfUajJzy8irBeywCmlObpV_IoVBVC0hmwQqzXe6XVjfdFbhvT1Ea9oRS8II8E58XQruIIfpWqY4Ojmhq
AT9NqM6kvzhytT9ToSncPD2VtaYg==
[2] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHe7PM1dGPL5qRXh2oNYwITkbE1bhUPaX61mXEi_9dWvTbO2EITJx8191_6SPSnJQmi7yAPYZk3w2szQ2a14JPQe
8jejE8Jhu-hiDJHtY_TdJH4yS4JJLg0T3PQ1FGH_k1Q0g==
[3] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGfR8PPOdqPw7oqI3Yk7LgovLDkQYHiN2IdwRdd8kn-Rn7Jp80E-
Hbk_zvlf_pW8K0LJsjrfLJogZZvQGWiFrDy9p0Zap7uGqPsrjrH0ZIzshnPsK5w48d7wJ9Cdjj2EcFvAb1HAw==
[4] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQE_SHI3m_jRVuORdkIXVZJnKB3rJeIooOrIs9nOB7giptIaNNQLVqHtRnwx38z2XXPtJWmfMJSK8xBKyIbxwZ2_m-
a1Y9d7e8gCN3seZXkRhDB98-HxZkR0gZtjOqUP3k_gISvzvg==
[5] wikipedia.org. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHrfI6X7IyX3HN8SXnWyFzDve9NnFJeSHbw64WtILVIOS7VQ3U60riEBFz0p_Yecbb-
uybguQdfT14986luzAtxFWNQYd-dU-rk4915ukQFUkvtXBwDcSe31NJg-XI26X5j
[6] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFWq9sGCII6I96Eh6pzL0BjyLViUMUNhrYTuwPBPOX51_SmBIDxuXIJDPPVEwNLzXRkLufqxPsN621C6M3DyLCHa
D9dIzcKAWpAZPIKIHOvQ=
[7] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFfcnmx8d7OM1IFwtxNeG87GYNjX21WWhfvWCMIhAeuvvP78KiIhzp69DSkIWNkhViNs4deSnX9zfu10YPmMC9S
VY=
[8] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFTo5nQ8VL4C7MibJTct-gh2R1ppjs9UvRrF7K5p025Ihn27ke0YJocpbAm4en5ZT8JrTIYGL10kxkaWIFyWXyB655xrY-ZAo-
4O656AenvgMRn29pJzjofF7kmxDc4SpC5vTI6iaF7IQ==
[9] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG2KOL2jBWofwe2PmOSdy4MfNsmNuQM2qRy36G6R3pQ_gljV9wDOLnIF5IMO08Ctd9_ZAeBVYeMP5L_O5Sdwt
[10] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFNcxfY6srdYAQWnI9PsoSPjR504VnS-JWorkt-
WOscIkqQrlXVQuDYA_NeFoXEEs4wn6uOyEVrTrZmhZqyv9I7cAn95fFvSt2plNGQz0uTV0AgsqpFJu-Xt-
uPLoa14QkCmvcLIxFsJOp8
[11] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGA1JSQeE70odZ14iOeXcMF939IKjBNGHrV5qrE77F-
FViVu3irLI0jjEsU5Aan55ZHOTEz7uEHEc3Zooz0MStc636Zh54w2S6M9Dax19jDVwy-
D9GrKPwnikS4Yj_1Zxb9ifXcXIU=
[12] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQH36wf0tWJjNo2fL_bGaDdJitRfi1P0vzFscnwIw0h54k4kCcprtbGsxSJXNeOwSgrIvRhNEV9oAVOLQcnoDaBpoTF4
RPEwiaTmvg0Xmmq4qy48GOw==
[13] wordpress.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEeL8v63Hu0ra9NeXbHbwJo0qvsr-
aURq8gv1lB3hdEQQhgvijUi6SJxxC2kekfBzcDJKOB_HjkKY8TqoaVNeO1HeukOJd6-8gzKL18qY8dGbXCzRI-q-
YteaVdF-bORWtOnArMtomzeu9hUoWdqYmEt4YyjDcJ03I3MInOXC0g69EwGaKT0S3Vo08mdkKAI_4rZu8=
[14] reddit.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEP1mpOEpH0Zh3eII63q9PhH6dt1xasRoHP-ztpoHnAy54kcE1CF4vXyQ4-
vwj6gaykYgrcpubl_4nVDS6rPKqwSwTb3T0h9mXgIKIB-k9S-
6NCG980R5M3dM6TCdtKgdiBpFwLmMTfIFr5HYcqbO1WvRQuLIL1aSU4MQQJgUfeOuqiQwtsKb1C_ysjFQdcXyEuhyiI4twZf5BjS6
GQ=
[15] reddit.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFcW0dOMacFQjQ5dWjB7DHUGmjST2eKOHyQu8gVIx5q3as5tTV9PbfmKnAu0q5eOr8YfGJ-
B3UO6hwY4Iv0FXEN2t0siOYsTt8EadNjGjv7nw7_5CpMB8aypoerBM6Xpena4IE3RlfGqsFyOtkpx-
HaJmQG6G3fverOnP88zbum3OSfxMr73I2LKluyPRLZwEI8C90RPwLGW-7A
[16] github.io. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQH_EHDX8dABBRD5SuncFwaGZ00fs2xe3b3vo1w52NC5Wb1Y3XThuhP6NdkGbwGG4aUeIkD4H0VD-
C63E7ApQQ82X2iKaWdU8NWbIob_IeT0KLOL533UxqshEOEKCyM4DZeFYDyEkDI-01_j
[17] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGKG6A_oTuge-SD76SWhEPsf_7IIfVCef5QYoCJQNpMSJ_T_-F_M5ftsiLyccaONVIKuwPJ4u9MEWC-
FbQhtnacb2UAyIBJncSdy2769QO31FaEWG1JwE1GVeHuUupXzuqMtAcQusHdIzerDKEq6-3q_VZqumqaizCE

[18] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHDv6Zc53j7UUlqrR1lJi2r9xSdQ1yFa2Iun4MxDnhSAhjDQBkyLKQ0019q83cHfEQscmhStveZzR3WKKWt8RUI6dq
WA0YX1iw2dKeYzt92JJf4mfBsZJE2K58ln6Z4v8zpEZmCd0TAALQiWvMZ4jgouuH0guVe_HIt
[19] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHhGuqSOMAmd3OGyRR7cC87Yp6UzKqqM_UPN1eIHA4VuXCUTsDR7MbVHxJvwy9y0OfEn0DHzfM3QOZkaiiz_
aRFBDod4yxpsqlN1Xv
[20] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG8D6ILo0IrItcgSPtxKToTHYmqK7aW464vIyqLXN1S-5Iuah5033tDLUQMcySvJqG2ELNFRQ1rzu8e8ww8H2Gx347d6OZFnTSx3-1Z2jRqy897xu0pliZE5Ockr38dAjveWCJK-1ZBzY0J1gV5ybk1PLLFJSmQE0pg3DEi9_J6oMrSO5XJZSIdaEDORpZQbnX3VUEJosDsKKjZ
[21] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQETH5It4i432kI-3aOXzfDROuuLqbvWtSHne0uemiHaSZADIuUfMRqlLzL2EN3YvpmVpBbaU0bEDZXhhESc-lw9XN4Wcajk_uP13EibCmdrXbpj74GNhtbPu4_qmtEUPsr0K-fhFHzLHutbT3IQrFV6QdxeUX4A0YVsbYQYpDeOtlpL7xnha2a8d6K8yhQRZS6bFeUUnwMAP_4v
[22] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQF7LhFoAEgKnvfN21QvOGwbCfddtBVE2tZAUSCbqcIAZYScI4kq8UajYcPDPk4zAGnk1taSj9mc1FJm1A2eOTweuHy
Y3STRto4gJZf7XAY=
[23] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHMofHN_FhZteuoHdBSvpNiegDG9uH_5ewr0La9xZx3i1t4Wv66EiSehtEKpIO9DS1ywjwtiMegWeVj13kY0MWPzo3
5mUOiWG4pfz3JrZwViTgr2x-3bVITCHqWTJBAp7r0xXihLDRqIUmjIK8wQrStkobXg65_OOpzD7HeYx52yExe42GnVjIC9vIntgmTMxUt1QDh2MVb2bNRJuZwLUd
[24] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFJ682HI82VdvIe7KrtnKNLdshKFRIyHP03LqL_2FBPEdvIkz0m6RDodO2JH0iA_sk49GZ32rq8AE0rzgpEQEkOdAJ
wai_WI5TA0ZeYnZypRIuGiuyOfoSAHCjN7TLHIdPNrIh7ZdIV9_gY2yDtW0E6Dt4nBYdZ_LkAD0w3QFqMWyGMogMGgoamQIM_po
[25] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQELtUcQt533_xxmMUOK3zS5H6F_Q1uY-gCPIeBDAYzhjyABKBJjetQdSPfnaenkCcDKgJA2ILuvT0OLfsW9aMgqmYjVi4eZkJFXLN2Bv2cGTrDZpfZxjfP0_EYMtb6E0e39KpqF
ApCXI-i_LW0Hh3A4kEA8Iv5N2zW_asILzrJs85k5xeAgpCMQzJgJPYjokIAHyH71gyQSi6sZy-2k=
[26] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG9eSafzCS9yXQYoqb4eeijyj1y37p40cKBxVT_Xu3_yxwJ4m9rJLfW1rm5K3vIuXRbAFKVNxZVS-C17vAf7D8QdGR6ttqWcsePrnQ3LUbmEQ659kahEGxAMaDEhoQNcLXWTLm0xaLS8nJInd98-f3KwOEm8K210qAigOJBsJVKtR5erO4IvhgXMwo7HhU=
[27] oracle.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHUW3INTsZIFjKbBPh0IT-rQ_ULZU9V3kDskQu1TqT6XbqgURMvnNc7z6fwbEx0Apk49gq1yYs_wk-kqHLaCCQSN6FUEiQw8yNscnxO5IwunPwRBxEd-XpIL4f94QteYgvJxRU3FK2GiIR45IEiNwnDFQEIPrVZv_8JpbhpTuMWn4OM-mErKQ==
[28] ubuntu.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQE6VpI9VyYd1htJ3Wc5XWzoVI3dfnRU0GPSazeHrnV4SGw2Q_vDKHbPS8uj-gSIKARb4urtBCkCErWqzkVG3xW-fIx_Sla8ieb2tioL3L5kISfACMrGpVmajp6ay84qJJa4Uzr0hHBqIh5gK9BNAeUAHNtR0IBDQniXPgykd-E=
[29] nvidia.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQH803Sh4ITIYLIdxPK-Bya2PIN8ioHccRE4m4CeQMnwb23RfAUpO5hQm0sIYqZh8ErmXMdXdlIV6k_Pa6sbOpBJWUOT9elZJAPjtF4D17p6gnQ_NEKuHffk
9tNyT5AZaghh3ECAqgSx6xQUphTEXto57fcXwg==
[30] nvidia.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFM2NY3RI8WvxHI6u1_CDb1NFRJRIzaBskmO7ZNZn0rpjeV4w4DkbF2M0AgOD9IYT5D4uvza3uQcjwLVxbtqw4rF
iDCyPS9kyyzi6rJAGdsR54JEq9HBTQJJ58xU4BSiEczxCv6N2HcIpGZ-1VIBE

Methodology

- **Backend**: VERTEX API

- **Model**: gemini-2.5-pro

- **Research Depth**: 3 (follow-up iterations)

- **Research Breadth:** 3 (parallel queries per iteration)
- **Total Sources Evaluated:** 103
- **Quality Threshold:** 40%+ (verified sources only)
- **Duration:** 352.9s
- **Synthesis Method:** AI-assisted cross-referencing and verification

---

API Costs

Metric	Value
Input Tokens	32,955
Output Tokens	9,905
Total Tokens	**42,860**
Input Cost	$0.0412
Output Cost	$0.0990
Total Cost	**$0.1402**

Report generated by ez-deep-research MCP